Algorithm Design

LAN-Based Secure Chatting Application

TE Computer Engineering Mini Project

Prepared by

Amogh Sirsikar

Aryan Patil

Malhar Nimbalkar

1. Purpose

This document specifies the core algorithms of the LAN-Based Secure Chatting Application, as derived from the project SRS. For each algorithm, the Input, Processing/Logic, and Output are stated explicitly, in addition to the step sequence, so that the design intent (not merely the runtime flow) is clear.

---

2. User Registration Algorithm

Input:

- Username (string)
- Password (plaintext, entered by user)

Processing / Logic:

1. Receive username and password from the client.

2. Validate that both fields are non-empty and meet basic format rules.

3. Query USERS table for an existing row with the same username.

4. If a match exists, reject the request with a duplicate-username error (FR-02) and stop.

5. If no match exists, hash the password using a one-way hashing algorithm (SEC-04).

6. Insert a new row into USERS with username, password_hash, status = 'offline', and created_at = current timestamp (FR-01).

7. Return a success acknowledgement to the client.

Output:

- Success: new user_id created and confirmation sent to client.
- Failure: duplicate-username or validation error message.

Complexity:

O(1) average for the uniqueness check, assuming an index/unique constraint on USERS(username) (Section 7.6).

---

3. User Login / Authentication Algorithm

Input:

- Username (string)
- Password (plaintext, entered by user)

Processing / Logic:

1. Receive username and password from the client.

2. Look up the USERS row matching the given username.

3. If no such user exists, return a generic authentication-failure message (FR-04) — do not reveal whether the username exists.

4. If the user exists, hash the entered password using the same algorithm used at registration.

5. Compare the computed hash with the stored password_hash.

6. If the hashes do not match, return the same generic authentication-failure message and stop.

7. If the hashes match, create a new Session object bound to this connection, set status = 'online' (FR-06), and record the session against the user (FR-12).

8. Return a login-success response together with the session state to the client.

Output:

- Success: authenticated session established, user marked online.

- Failure: generic error message, no session created.

Complexity:

O(1) average for the credential lookup, given a unique index on username.

---

4. Send Private Message Algorithm

Input:

- Sender's session (must be authenticated)
- Recipient user identifier
- Message text (string)

Processing / Logic:

1. Verify that the sender's connection has an active, valid session (SEC-02); reject if not authenticated.

2. Validate the message text: reject and notify the sender if it is empty (FR-26).

3. Identify or create the Conversation record between sender and recipient.

4. Persist the message in the MESSAGES table within a database transaction, storing conversation_id, sender_id, message_text, and timestamp (FR-18, FR-21).

5. Submit the message text to the PDS module for classification (FR-27); if the PDS module is unreachable, log the condition and continue with classification marked 'unavailable' rather than blocking delivery (FR-32, FR-39).

6. Update the persisted message with the returned classification probability and label (FR-30), inside the same transaction as the persistence step where practical (NFR-15).

7. Check whether the recipient's connection is currently active (online).

8. If the recipient is online, route the message and its classification to the recipient's connection (FR-17).

9. If the recipient is offline, skip live delivery; the message remains retrievable through normal chat-history lookup on the recipient's next connection (UC-05 alternative flow).

10. Return a delivery/status acknowledgement to the sender (FR-20).

Output:

- Message persisted in PostgreSQL with sender, conversation, and timestamp.
- Classification probability and label attached to the message.
- Delivery status returned to sender; message delivered live to recipient if online.

Complexity:

O(1) for routing and persistence per message (excluding PDS inference time, which is external to this algorithm).

---

5. View Chat History Algorithm

Input:

- Requesting user's session (must be authenticated)
- Conversation identifier

Processing / Logic:

1. Verify the requester holds a valid, authenticated session (SEC-02).

2. Verify the requester is a participant of the requested conversation (SEC-03); reject otherwise.

3. Query the MESSAGES table for all rows with the given conversation_id, ordered by timestamp.

4. If the database is unavailable, return a 'history temporarily unavailable' response rather than failing silently (FR-25).

5. Return the ordered list of messages (with classification labels, where present) to the client.

Output:

- Ordered list of messages for the conversation, or a clear temporary-unavailability message on DB failure.

Complexity:

O(k log k) or O(k) depending on whether MESSAGES(conversation_id) is indexed and pre-sorted, where k is the number of messages in the conversation (Section 7.6).

---

6. Message Classification Algorithm (Logistic Regression)

Input:

- Raw message text (string)
- Trained model parameters (weight vector w, bias b)
- Configurable classification threshold (default 0.5)

Processing / Logic:

1. Extract a feature vector x from the message text (e.g., TF-IDF/term-frequency values, message length, suspicious-term indicators) (PDS-01).

2. Compute the linear combination z = w·x + b.

3. Apply the sigmoid function to obtain a probability: p = 1 / (1 + e^-z) (PDS-02).

4. Compare p against the configurable threshold: if p >= threshold, assign label 'Potential Spam' or 'Potentially Misleading'; otherwise assign label 'Normal' (PDS-03, FR-29).

5. Package the result as {probability: p, label} and return it to the Java server (FR-30).

Output:

- A numeric probability p (0 to 1).
- A classification label derived from p and the threshold.
- Never presented to the user as a verified fact (FR-31, UI-5).

Complexity:

O(d) per message, where d is the number of extracted features; independent of message-history size.

---

7. Supporting Metric Estimation Algorithm (Linear Regression)

Input:

- Historical operational observations (e.g., timestamps used to derive message delivery latency) (FR-33)
- Trained model parameters (coefficients)

Processing / Logic:

1. Collect the relevant measurable observations for the chosen supporting metric during system operation.

2. Construct the feature vector for the current observation.

3. Compute the estimated metric value as a linear combination of the input features and trained coefficients (FR-34).

4. Return the estimated value alongside the actual observation, where available, for later evaluation.

5. During evaluation (not at runtime), compare predicted vs. actual values across the test set to compute MAE and RMSE (FR-35).

Output:

- A numeric estimate of the chosen supporting metric.
- MAE and RMSE scores, computed from actual test data during evaluation.

Complexity:

O(d) per estimate, where d is the number of input features.
